Analítica de Datos y Herramientas de Inteligencia Artificial

Actividad 5 (Evaluación)

Regresión lineal simple, múltiple y logística

Reporte de la actividad

Ian Kaled Sandoval Gines
Matrícula A01738687

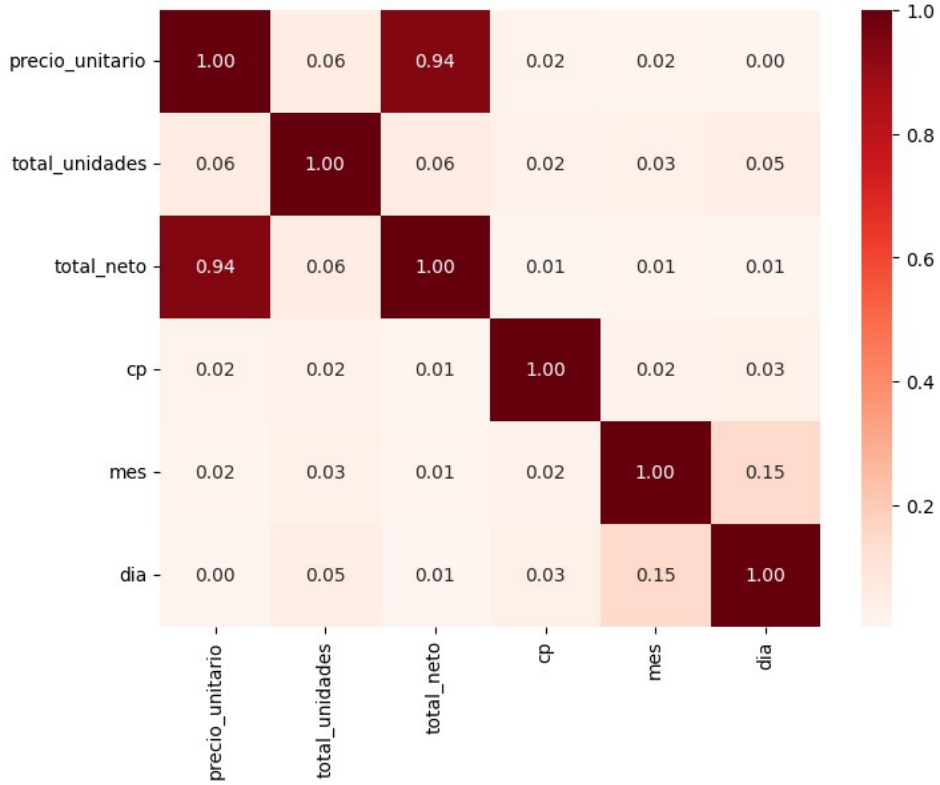
Prof. Alfredo García

19 de octubre de 2026

Se trabajó con la hoja Histórico_Ventas del archivo Datos_Complementarios_CLV.xlsx (21,632 ventas). Antes de modelar se limpiaron los nombres de columna, se unificaron los estados, se sustituyeron los nulos de cp y las ventas en cero, y se trataron los valores atípicos por desviación estándar. El detalle de todo el preprocesamiento está en el notebook Actividad_5.ipynb; este reporte se centra en los modelos.

1. Regresión lineal simple

Se calculó la matriz de correlación de las variables numéricas. La correlación más alta de la base es entre precio_unitario y total_neto, con 0.94; las demás variables (cp, mes, dia) no se relacionan con nada.



Mapa de calor de los coeficientes de correlación

Con total_neto como variable dependiente y precio_unitario como independiente se ajustó la regresión lineal simple.

```
#Verificamos los coeficientes obtenidos para el modelo ajustado
model.__dict__
```

```
{'fit_intercept': True,
 'copy_X': True,
 'tol': 1e-06,
 'n_jobs': None,
 'positive': False,
 'feature_names_in_': array(['precio_unitario'], dtype=object),
 'n_features_in_': 1,
 'coef_': array([0.98489506]),
 'rank_': 1,
 'singular_': array([1444696.82725295]),
 'intercept_': np.float64(1055.507449461103)}
```

```
#Armamos la ecuación del modelo
print("total_neto =", model.coef_[0], "* precio_unitario +", model.intercept_)
```

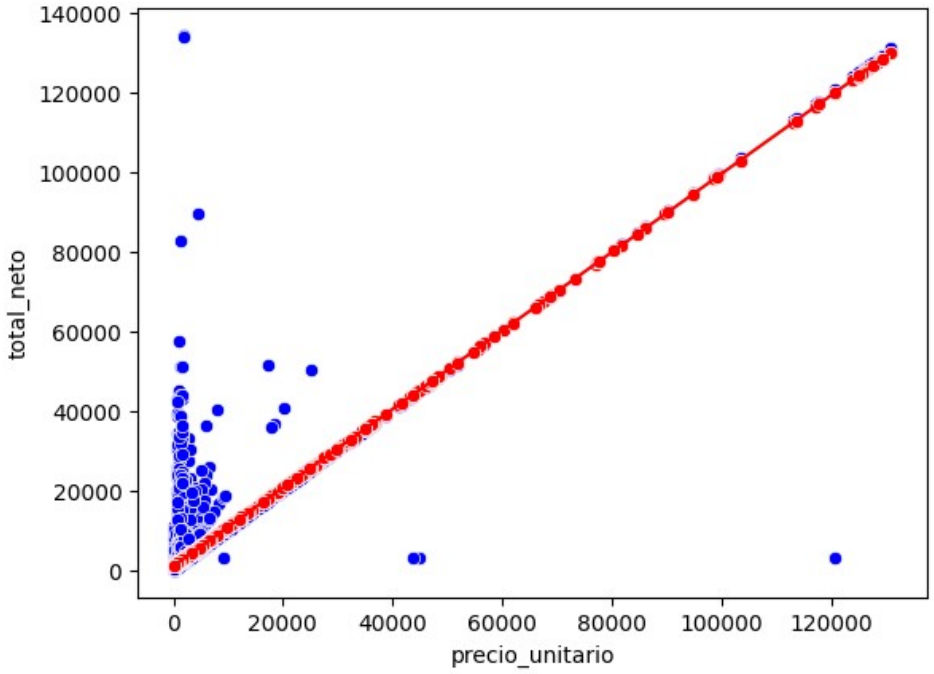
```
total_neto = 0.9848950572995678 * precio_unitario + 1055.507449461103
```

Coeficientes obtenidos con el modelo simple

Modelo matemático obtenido:

$$total_neto = 0.9849\ (precio_unitario) + 1055.51$$

El coeficiente de determinación R^2 fue de 0.8902 y el de correlación r de 0.9435: el precio unitario explica alrededor del 89% de la variación del total neto. Los puntos que quedan por arriba de la línea son las ventas de varias unidades, donde el precio solo ya no explica el total.



Total neto real (azul) contra predicción del modelo simple (rojo)

2. Regresión lineal múltiple

Se probaron varias combinaciones de variables para explicar total_neto. Agregar mes, dia y cp prácticamente no mueve el R^2 (de 0.9039 a 0.9042), así que el mejor modelo múltiple se quedó con precio_unitario y total_unidades.

```
#Armamos la ecuación del modelo
print("total_neto =", model_m.coef_[0], "* precio_unitario +", model_m.coef_[1], "* total_unidades +",
```

```
total_neto = 0.9924928018455826 * precio_unitario + 484.8695340542321 * total_unidades + -36.919673822
```

```
#Evaluamos la eficiencia del modelo obtenido por medio del coeficiente R Determinación
coef_Deter_m = model_m.score(X=Vars_Indep_m, y=Var_Dep_m)
coef_Deter_m
```

```
0.9039206892717127
```

```
#Corroboramos cual es el coeficiente de Correlación de nuestro modelo
coef_Correl_m = np.sqrt(coef_Deter_m)
coef_Correl_m
```

```
np.float64(0.9507474371628423)
```

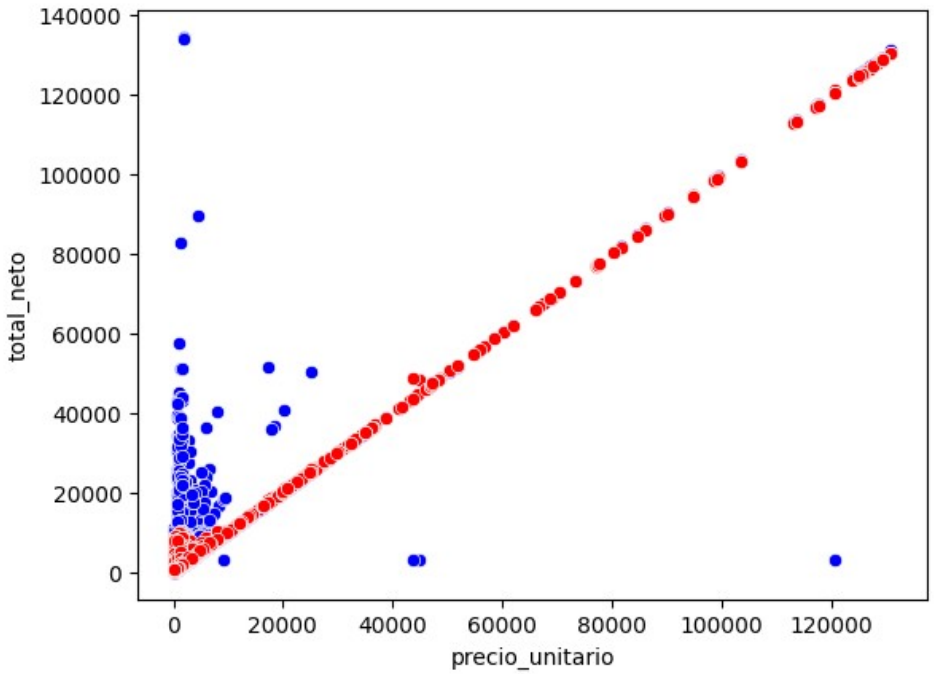
Ecuación y métricas del modelo múltiple

$$total_neto = 0.9925\ (precio_unitario) + 484.87\ (total_unidades) - 36.92$$

Comparación con el modelo simple

	Regresión simple	Regresión múltiple
Coef. precio_unitario	0.9849	0.9925
Coef. total_unidades	—	484.87
Intercepto	1055.51	−36.92
R^2 (determinación)	0.8902	0.9039
r (correlación)	0.9435	0.9507

- El coeficiente de precio_unitario casi no cambia (0.985 contra 0.992): cada peso de precio se refleja casi en un peso de total en los dos modelos.
- El intercepto cambia mucho, de 1055.51 a −36.92. En el modelo simple estaba absorbiendo el efecto de las unidades; al agregar total_unidades ese efecto pasa a su propio coeficiente (484.87 pesos por unidad extra) y el intercepto se acerca a cero.
- El R^2 sube de 0.890 a 0.904 y r de 0.944 a 0.951. La mejora es pequeña pero real: la segunda variable explica justo las ventas de varias unidades que el modelo simple no alcanzaba.



Total neto real (azul) contra predicción del modelo múltiple (rojo)

3. Regresión logística

La columna estado se convirtió en dicotómica: CDMX contra Otro estado (4,315 contra 17,317 ventas, desbalanceada). Como variables independientes se usaron precio_unitario, total_unidades y total_neto, con 70% de los datos para entrenar y 30% para probar.

Matriz de confusión del modelo sin balancear

```
#Definimos el algoritmo a utilizar
from sklearn.linear_model import LogisticRegression
algoritmo = LogisticRegression()

#Entrenamos el modelo
algoritmo.fit(X_train, y_train)
```

```
LogisticRegression()
```

```
#Verifico la matriz de Confusión
from sklearn.metrics import confusion_matrix
matriz = confusion_matrix(y_test, y_pred)
print('Matriz de Confusión:')
print(matriz)
```

```
Matriz de Confusión:
[[   0 1319]
 [   1 5170]]
```

```
#Calculo la precisión, exactitud, sensibilidad y F1 del modelo
from sklearn.metrics import precision_score, accuracy_score, recall_score, f1_score

precision = precision_score(y_test, y_pred, average="binary", pos_label="CDMX")
exactitud = accuracy_score(y_test, y_pred)
sensibilidad = recall_score(y_test, y_pred, average="binary", pos_label="CDMX")
puntajef1 = f1_score(y_test, y_pred, average="binary", pos_label="CDMX")

print('Precisión del modelo:', precision)
print('Exactitud del modelo:', exactitud)
print('Sensibilidad del modelo:', sensibilidad)
print('Puntaje F1 del modelo:', puntajef1)
```

```
Precisión del modelo: 0.0
Exactitud del modelo: 0.7966101694915254
Sensibilidad del modelo: 0.0
Puntaje F1 del modelo: 0.0
```

Regresión logística sin balancear: matriz de confusión y métricas

Las filas son el valor real y las columnas la predicción, con las clases en orden alfabético (CDMX, Otro estado):

	Predijo CDMX	**Predijo Otro estado**
Real CDMX	0 (verdaderos positivos)	1,319 (falsos negativos)
Real Otro estado	1 (falsos positivos)	5,170 (verdaderos negativos)

A partir de la matriz salen las métricas:

- Exactitud = (0 + 5,170) / 6,490 = 0.7966. Parece buena, pero es solo porque el 80% de las ventas de prueba son de otros estados.
- Precisión (CDMX) = 0 / (0 + 1) = 0. De lo que marcó como CDMX, nada lo era.
- Sensibilidad (CDMX) = 0 / (0 + 1,319) = 0. No detectó ni una venta de CDMX.
- F1 = 0, porque combina precisión y sensibilidad y las dos son cero.

El modelo aprendió a decir siempre "Otro estado", por eso la exactitud engaña. Para corregirlo se aplicaron reponderación de clases (class_weight='balanced') y oversampling (SMOTE).

Comparación con los modelos balanceados

Métrica	Sin balancear	Reponderación	Oversampling
Precisión	0.000	0.262	0.262
Exactitud	0.797	0.490	0.490
Sensibilidad	0.000	0.834	0.832
F1	0.000	0.399	0.399

Con reponderación la matriz queda [[1,100, 219], [3,092, 2,079]]: ahora acierta 1,100 de las 1,319 ventas de CDMX (sensibilidad 0.834), pero clasifica como CDMX 3,092 ventas de otros estados, por lo que la precisión baja a 0.262 y la exactitud a 0.490. SMOTE da prácticamente lo mismo. Los dos métodos logran que el modelo detecte el 83% de las ventas de CDMX, cosa que el original no hacía, aunque a costa de la precisión: solo con precio, unidades y total es difícil distinguir de qué estado viene una venta.

Segunda variable dicotómica: marca

También se convirtió marca en VITA contra Otra marca y total_neto en Venta baja / Venta alta (corte en la mediana con pd.cut), y se corrió otra regresión logística balanceada para predecir la marca.

	Predijo Otra marca	Predijo VITA
Real Otra marca	2,658	1,122
Real VITA	299	2,411

Aquí el modelo funciona mucho mejor: exactitud de 0.781, sensibilidad de 2,411 / (2,411 + 299) = 0.890 para VITA, precisión de 2,411 / (2,411 + 1,122) = 0.682 y F1 de 0.772. Tiene sentido que el precio y las unidades sí ayuden, porque cada marca maneja rangos de precio distintos.

4. Conclusiones

- El modelo lineal simple total_neto = 0.9849(precio_unitario) + 1055.51 explica el 89% de la variación del total (r = 0.94).
- El modelo múltiple con precio_unitario y total_unidades sube el R^2 a 0.90 y deja un intercepto cercano a cero; mes, día y cp no aportan nada.
- Predecir si una venta es de CDMX no funciona sin balancear (sensibilidad 0); con reponderación u oversampling se detecta el 83% de las ventas de CDMX, perdiendo precisión. Predecir la marca sí funciona bien, con 78% de exactitud y F1 de 0.77.